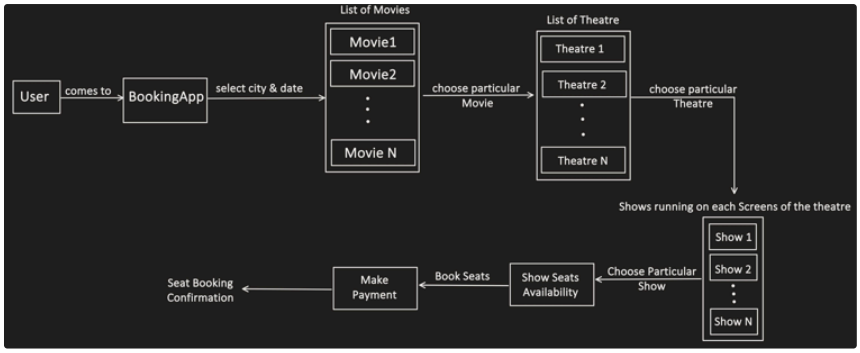
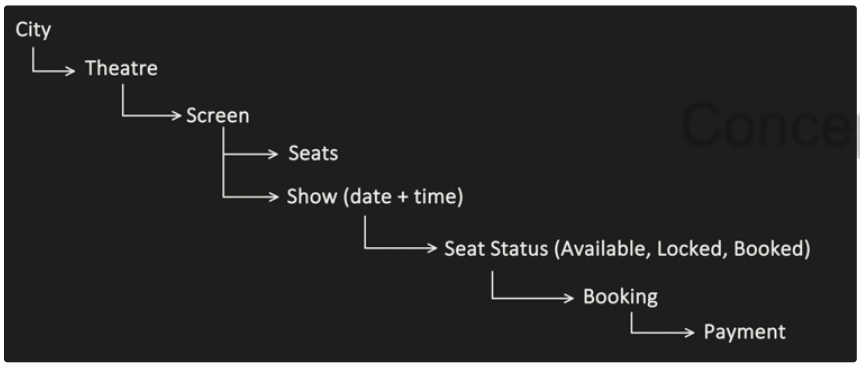
Design Movie Ticket Booking App

Reference: Concept&&Coding-By Shrayansh
LLD of BookMyShow (English) | Design MovieTicketBooking

Lets Understand the flow:

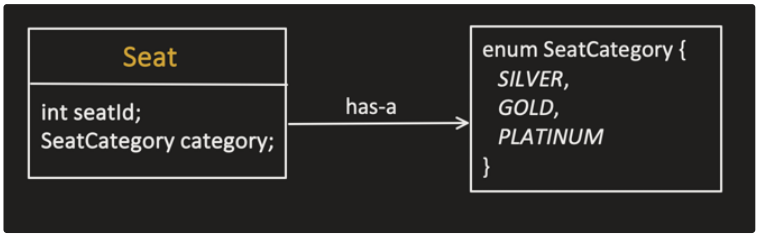




Concept && Coding

UML:

1. Seat

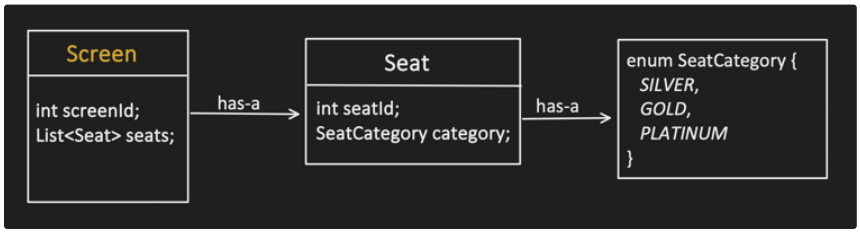



```java
public class Seat {

    private final int seatId;
    private final SeatCategory category;

    public Seat(int seatId, SeatCategory category) {
        this.seatId = seatId;
```

```java
        this.category = category;
    }

    public int getSeatId() {
        return seatId;
    }
}
```

```java
public enum SeatCategory {
    SILVER,
    GOLD,
    PLATINUM
}
```

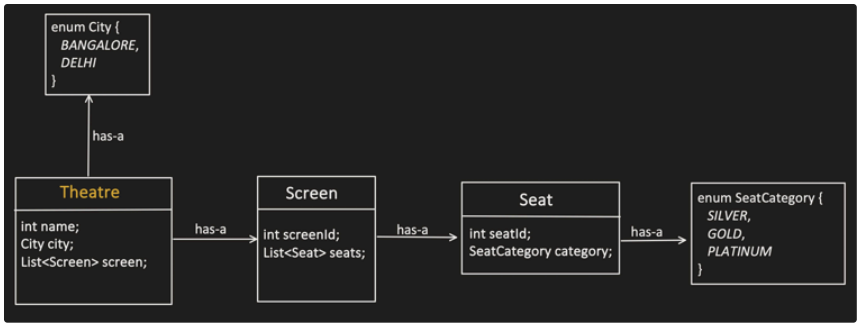
2. Screen




```java
public class Screen {

    private final int screenId;
    private final List<Seat> seats;
    private final Map<LocalDate, List<Show>> showsByDate = new HashMap<>();

    public Screen(int screenId, List<Seat> seats) {
        this.screenId = screenId;
        this.seats = seats;
    }

    public List<Seat> getSeats() {
        return seats;
    }

    public void addShow(Show show) {
        showsByDate
                .computeIfAbsent(show.getShowDate(), d -> new ArrayList<>())
                .add(show);
    }

    public List<Show> getShows(LocalDate date) {
        return showsByDate.getOrDefault(date, List.of());
    }
}
```

Concept && Coding

3. **Theatre**

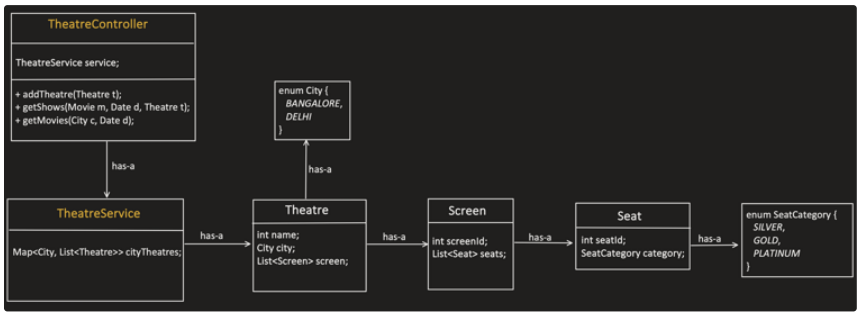



```java
public class Theatre {

    private final String name;
    private final City city;
    private final List<Screen> screens;
    //address info etc.

    public Theatre(String name, City city, List<Screen> screens) {
        this.name = name;
        this.city = city;
        this.screens = screens;
    }

    public City getCity() {
        return city;
    }

    public String getName() {
        return name;
    }

    public List<Screen> getScreens() {
        return screens;
    }
}
```

Concept && Coding

```java
public enum City {
    BANGALORE,
    DELHI
}
```

4. **Theatre Controller**



TheatreController:

```java
public class TheatreController {

    private final TheatreService theatreService;

```

```java
    public TheatreController() {
        this.theatreService = new TheatreService();
    }

    public void addTheatre(Theatre theatre) {
        theatreService.addTheatre(theatre);
    }

    public Set<Movie> getMovies(City city, LocalDate date) {
        return theatreService.getMovies(city, date);
    }

    public List<Theatre> getTheatres(City city, Movie movie, LocalDate date) {
        return theatreService.getTheatres(city, movie, date);
    }

    public List<Show> getShows(Movie movie, LocalDate date, Theatre theatre) {
        return theatreService.getShows(movie, date, theatre);
    }
}
```

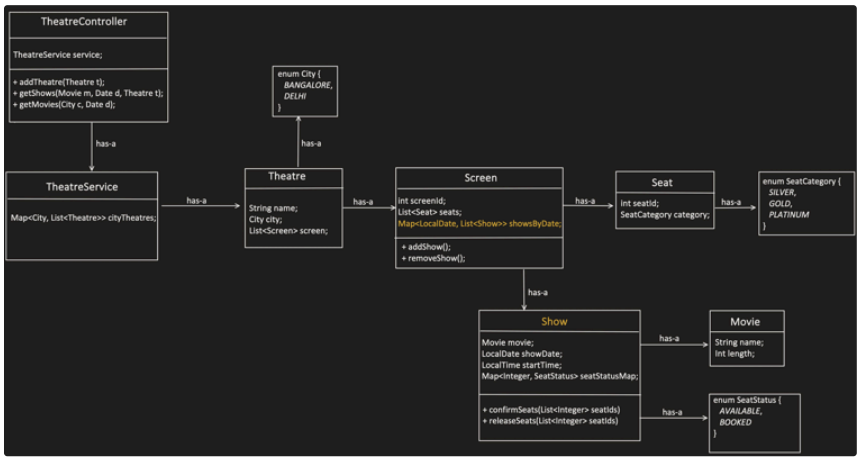
TheatreService:

```java
public class TheatreService {

    private final Map<City, List<Theatre>> cityTheatres = new HashMap<>();

    public void addTheatre(Theatre theatre) {
        cityTheatres
                .computeIfAbsent(theatre.getCity(), c -> new ArrayList<>())
                .add(theatre);
    }

    public Set<Movie> getMovies(City city, LocalDate date) {
        Set<Movie> movies = new HashSet<>();
        List<Theatre> theatres = cityTheatres.getOrDefault(city, List.of());

        for (Theatre theatre : theatres) {
            for (Screen screen : theatre.getScreens()) {
                for (Show show : screen.getShows(date)) {
                    movies.add(show.getMovie());
                }
            }
        }
        return movies;
    }

    public List<Theatre> getTheatres(City city, Movie movie, LocalDate date) {
        List<Theatre> theatres = cityTheatres.getOrDefault(city, List.of());

        return theatres.stream()
                .filter(t -> t.getScreens().stream()
                        .anyMatch(s -> s.getShows(date).stream()
                                .anyMatch(show -> show.getMovie().equals(movie))))
                .toList();
    }

    public List<Show> getShows(Movie movie, LocalDate date, Theatre theatre) {
        List<Show> result = new ArrayList<>();
```

Concept && Coding

```java

        for (Screen screen : theatre.getScreens()) {
            for (Show show : screen.getShows(date)) {
                if (show.getMovie().equals(movie)) {
                    result.add(show);
                }
            }
        }
        return result;
    }
}

```

5. Show



Concept && Coding

```java
public class Show {

    private final Movie movie;
    private final LocalDate showDate;
    private final LocalTime startTime;

    private final Map<Integer, SeatStatus> seatStatusMap = new HashMap<>();
    private final Map<Integer, ReentrantLock> seatLocks = new HashMap<>();

    public Show(Movie movie, Screen screen, LocalDate date, LocalTime time) {
        this.movie = movie;
        this.showDate = date;
        this.startTime = time;

        for (Seat seat : screen.getSeats()) {
            seatStatusMap.put(seat.getSeatId(), SeatStatus.AVAILABLE);
            seatLocks.put(seat.getSeatId(), new ReentrantLock());
        }
    }

    public Movie getMovie() {
        return movie;
    }

    public LocalDate getShowDate() {
        return showDate;
    }

    public LocalTime getStartTime() {
        return startTime;
    }

```

```java
    public boolean lockSeats(List<Integer> seatIds) {
        List<Integer> sorted = new ArrayList<>(seatIds);

        //sorting i am doing to avoid deadlock scenario
        Collections.sort(sorted);

        List<ReentrantLock> acquiredLocks = new ArrayList<>();

        try {
            // Phase 1: acquire all locks
            for (int seatId : sorted) {
                ReentrantLock lock = seatLocks.get(seatId);
                lock.lock();
                acquiredLocks.add(lock);
            }

            // Phase 2: validate availability
            for (int seatId : sorted) {
                if (seatStatusMap.get(seatId) != SeatStatus.AVAILABLE) {
                    return false;
                }
            }

            // Phase 3: mark LOCKED
            for (int seatId : sorted) {
                seatStatusMap.put(seatId, SeatStatus.LOCKED);
            }

            return true;

        } finally {
            // Phase 4: release locks
            for (ReentrantLock lock : acquiredLocks) {
                lock.unlock();
            }
        }
    }

    public void confirmSeats(List<Integer> seatIds) {
        for (int seatId : seatIds) {
            seatStatusMap.put(seatId, SeatStatus.BOOKED);
        }
    }

    public void releaseSeats(List<Integer> seatIds) {
        for (int seatId : seatIds) {
            seatStatusMap.put(seatId, SeatStatus.AVAILABLE);
        }
    }
}
```

Concept && Coding

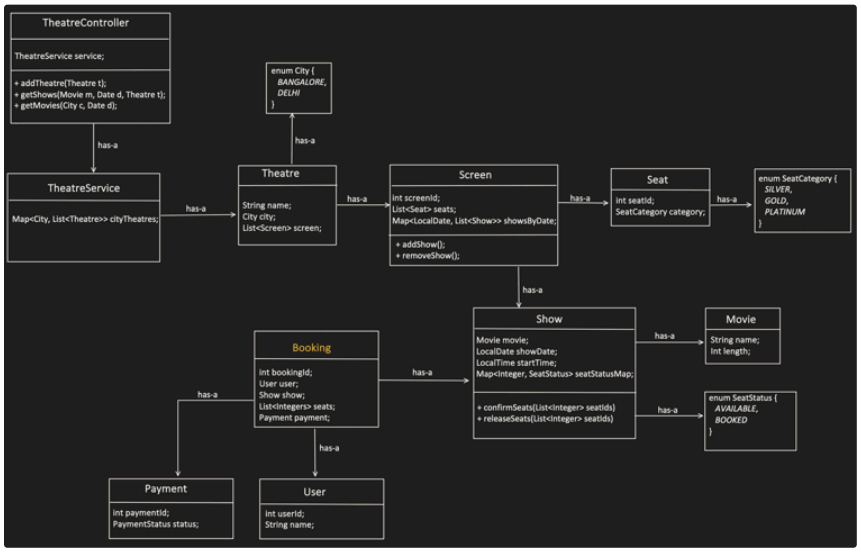
```java
public enum SeatStatus {
    AVAILABLE, LOCKED, BOOKED
}
```

Movie:

```java
public class Movie {

    private final String name;
    //duration

    public Movie(String name) {
        this.name = name;
    }
```

```
    public String getName() {
        return name;
    }
}
```

6. **Booking**




```
public class Booking {

    private final UUID bookingId;
    private final User user;
    private final Show show;
    private final List<Integer> seats;
    private final Payment payment;


    public Booking(User user, Show show, List<Integer> seats, Payment
payment) {
        this.bookingId = UUID.randomUUID();
        this.user = user;
        this.show = show;
        this.seats = seats;
        this.payment = payment;
    }

    public UUID getBookingId() {
        return bookingId;
    }

    public User getUser() {
        return user;
    }

    public Payment getPayment() {
        return payment;
    }
}
```

Concept && Coding

User:

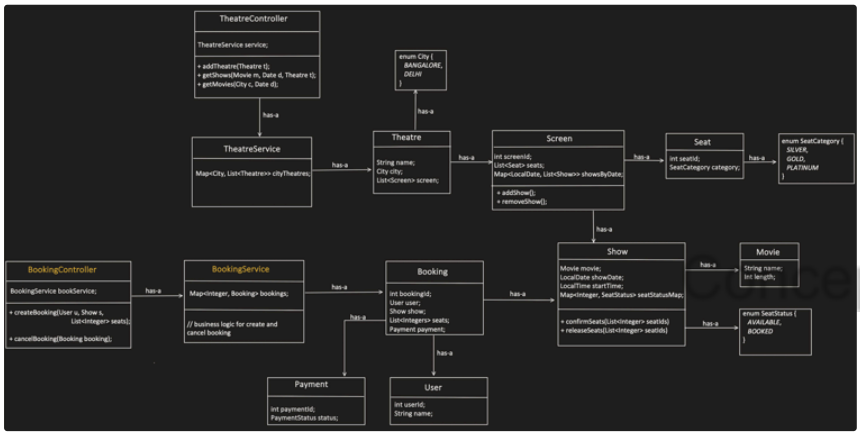
```
public class User {

    private final String userId;
    private final String name;
```

```
    public User(String userId, String name) {
        this.userId = userId;
        this.name = name;
    }
}

```

- if Interviewer says, we need to manage the User lifecycle too like add, remove ,delete. Then we can have its own controller like UserController. But here in booking functionality, we are not creating or managing user, so I am skipping exposing the endpoints for managing the lifecycle of User.

- Similarly, I am not creating PaymentController, PaymentService layer for now, but if interviewer want we can add that too.

7. **Booking Controller**



BookingController:

```
public class BookingController {

    private final BookingService bookingService;

    public BookingController() {
        this.bookingService = new BookingService();
    }

    public Booking createBooking(User user, Show show, List<Integer>
seats) {
        Booking booking = bookingService.book(user, show, seats);
        return booking;
    }

    public Booking getBooking(UUID bookingId) {
        return bookingService.getBooking(bookingId);
    }

    public List<Booking> getBookingsForUser(User user) {
        return bookingService.getBookingsForUser(user);
    }
}

```

BookingService:

```java
public class BookingService {

    private final Map<UUID, Booking> bookings = new HashMap<>();


    public Booking book(User user, Show show, List<Integer> seats) {

        if (!show.lockSeats(seats)) {
            throw new RuntimeException("Seat unavailable");
        }

        //simulated payment flow here, we can invoke Pay method of Payment Controller
        Payment payment = new Payment(PaymentStatus.SUCCESS);

        if (payment.getStatus() == PaymentStatus.SUCCESS) {
            show.confirmSeats(seats);
            Booking booking =  new Booking(user, show, seats, payment);
            bookings.put(booking.getBookingId(), booking);
            return booking;
        } else {
            show.releaseSeats(seats);
            throw new RuntimeException("Payment failed");
        }
    }

    public Booking getBooking(UUID bookingId) {
        return bookings.get(bookingId);
    }

    public List<Booking> getBookingsForUser(User user) {
        return bookings.values()
                .stream()
                .filter(b -> b.getUser().equals(user))
                .toList();
    }
}
```

Concept && Coding

Now lets talk about concurrency, 2 users should not book same seat.

So, what's the right place to put the lock:

1. Show?

It will work, but what if 2 users are booking totally different seats?

U1: S1, S2

U2: S4,S5

If we are putting the lock at Show level, then even users are booking the different seats, need to wait for the lock release.

2. Show Seat level?

This give us higher level of concurrency. But every important point is, we need to avoid deadlock scenario:

U1: S1, S5

U2: S5, S1

Now,:

U1 has acquired the lock on S1

U2 has acquired the lock on S5

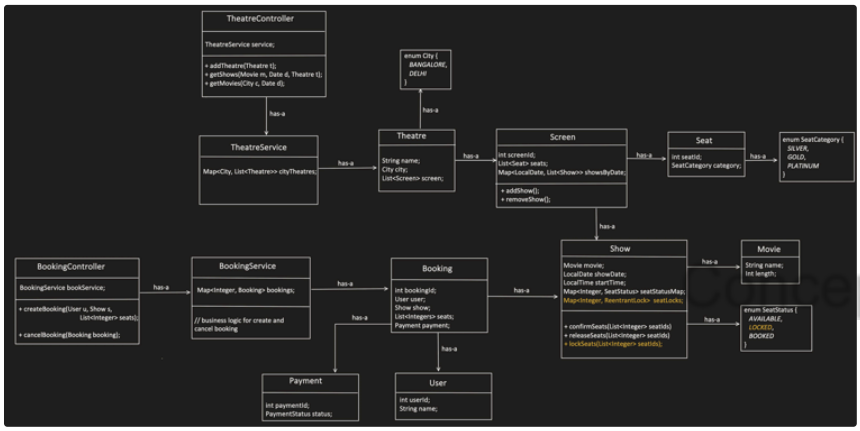
Now both are waiting for each other to release the lock on S5 and S1 respectively. Hence a deadlock scenario.

So, we can easily solve this issue through sorting of the Seat list.

U1: S1, S5 → Sorted → S1, S5

U2: S5, S1 → Sorted → S1, S5

Now, deadlock is not possible.



Client(BookMyShowApp):

```
public class BookMyShowApp {

    private TheatreController theatreController;
    private BookingController bookingController;

    public static void main(String[] args) {
        BookMyShowApp app = new BookMyShowApp();
        app.initialize();
        app.userFlow();
    }


    private void initialize() {
        theatreController = new TheatreController();
        bookingController = new BookingController();


        /*
         * 1. Create Movies
         */
        Movie baahubali = new Movie("BAAHUBALI");
        Movie avengers = new Movie("AVENGERS");


```

```java
        /*
         * 2. Create Theatre -> Screen -> Seats
         */
        Screen inoxScreen1 = new Screen(1, createSeats());
        Theatre inoxTheatreBangalore = new Theatre(
                "INOX",
                City.BANGALORE,
                List.of(inoxScreen1)
        );

        Screen pvrScreen1 = new Screen(1, createSeats());
        Theatre pvrTheatreDelhi = new Theatre(
                "PVR",
                City.DELHI,
                List.of(pvrScreen1)
        );

        theatreController.addTheatre(inoxTheatreBangalore);
        theatreController.addTheatre(pvrTheatreDelhi);


        /*
         * 3. Create Shows
         */
        Show inoxMorningShowToday = new Show(
                baahubali,
                inoxScreen1,
                LocalDate.now(),
                LocalTime.of(8, 0)
        );

        Show inoxAfternoonShowToday = new Show(
                baahubali,
                inoxScreen1,
                LocalDate.now(),
                LocalTime.of(15, 0)
        );

        Show inoxEveningShowToday = new Show(
                avengers,
                inoxScreen1,
                LocalDate.now(),
                LocalTime.of(18, 0)
        );


        Show pvrMorningShowTomorrow = new Show(
                baahubali,
                pvrScreen1,
                LocalDate.now().plusDays(1),
                LocalTime.of(9, 0)
        );


        // Attach shows to screens
        inoxScreen1.addShow(inoxMorningShowToday);
        inoxScreen1.addShow(inoxAfternoonShowToday);
        inoxScreen1.addShow(inoxEveningShowToday);
        pvrScreen1.addShow(pvrMorningShowTomorrow);
    }

    /*
     * USER FLOW (END TO END)
     */
    private void userFlow() {

        // User enters system
        User user = new User("U1", "Shrayansh");

        System.out.println("User logged in: Shrayansh");

```

Concept && Coding

```java
        // 1. User selects city
        City selectedCity = City.BANGALORE;
        System.out.println("Selected City: " + selectedCity);

        // 2. for specific date, Show movies running in city
        LocalDate selectedDate = LocalDate.now();
        System.out.println("Selected Date: " + selectedDate);

        Set<Movie> movies = theatreController.getMovies(selectedCity, selectedDate);
        System.out.println("Movies available:");
        movies.forEach(m -> System.out.println(" - " + m.getName()));

        // 3. User selects movie
        Movie selectedMovie = movies.iterator().next(); //selecting first movie
        System.out.println("Selected Movie: " + selectedMovie.getName());


        // 4. Show theatres and show times in city
        List<Theatre> theatres = theatreController.getTheatres(selectedCity, selectedMovie, selectedDate);
        System.out.println("Theatres available:");
        theatres.forEach(t -> System.out.println(" - " + t.getName()));

        // 6. User selects theatre
        Theatre selectedTheatre = theatres.get(0);
        System.out.println("Selected Theatre: " + selectedTheatre.getName());

        // 7. Show running shows for movie + date + theatre
        List<Show> shows =
                theatreController.getShows(
                        selectedMovie,
                        selectedDate,
                        selectedTheatre
                );

        System.out.println("Shows available:");
        shows.forEach(s ->
                System.out.println(" - " + s.getStartTime())
        );

        // 8. User selects show
        Show selectedShow = shows.get(0);
        System.out.println("Selected Show Time: " + selectedShow.getStartTime());

        // 9. User selects seats
        List<Integer> selectedSeats = List.of(1, 2, 3);
        System.out.println("Selected Seats: " + selectedSeats);

        // 10. Booking + Payment
        Booking booking =
                bookingController.createBooking(
                        user,
                        selectedShow,
                        selectedSeats
                );

        System.out.println("BOOKING SUCCESSFUL");
        System.out.println("Booking ID: " + booking.getBookingId());
    }

    private List<Seat> createSeats() {
        List<Seat> seats = new ArrayList<>();
        for (int i = 1; i <= 20; i++) {
            seats.add(new Seat(i, SeatCategory.SILVER));
```

Concept && Coding

```
        }
        return seats;
    }
}
```

Concept && Coding